

Exp. 6.3 Exercise: Critical Essay

Theme: Automatically Classifying Issues

Observations

- Both representative studies from the classical ML era, Kallis et al. (2019) and Pandey et al. (2017), targeted the same three-category taxonomy of Bug, Feature, and Enhancement, suggesting a community consensus around this basic classification schema during this period.
- Kallis et al. (2019) achieved 73 to 82% accuracy using FastText embeddings across both GitHub and Jira datasets, while Pandey et al. (2017) achieved approximately 73% accuracy using Naïve Bayes, SVM, and Random Forest classifiers, with SVM and Random Forest outperforming Naïve Bayes.
- Across both studies, textual features drawn from issue titles and bodies were identified as the most predictive signals, and both works demonstrated improvements over prior keyword-based approaches.

Critique/Implications for Future Research

The convergence on a three-category taxonomy across both studies is a convenient simplification that may not reflect the complexity of real-world issue trackers, where categories such as security vulnerabilities, documentation requests, and performance regressions are common but absent from the classification targets. This narrow framing inflates the apparent utility of these systems. Furthermore, while Kallis et al. tested their model across both GitHub and Jira datasets (a methodological strength), the modest accuracy ceiling of around 73 to 82% suggests that bag-of-words and shallow embedding approaches may be approaching their limits on this task. Future research should expand the label taxonomy to better reflect practitioner needs and explore whether richer contextual signals, such as code diffs linked to the issue or repository metadata, could push accuracy beyond this plateau.

Theme: Automatically Assigning Issues

Observations

- Across the seven classical ML studies summarized in Exploration 6.3, precision on the issue assignment task ranged from 57% to 65%, with Anvik et al. (2006) reporting the lowest precision at 57% and Kagdi et al. (2012) reporting the highest at 65%, indicating that performance improvements over the 2006 to 2012 period were incremental rather than transformative.
- Jeong et al. (2009) introduced a graph-based semi-supervised learning approach that modeled bug reassignment chains, referred to as tossing graphs, to reduce the phenomenon of issues being repeatedly reassigned before reaching the right developer.
- Xuan et al. (2017) reported a notably higher accuracy of 86% by incorporating incremental learning, meaning the model was updated as new assignments occurred, and demonstrated that this approach substantially reduced reassignment rates compared to static models.

Critique/Implications for Future Research

The striking difference between Xuan et al.'s 86% accuracy and the 57 to 65% precision range of earlier studies is difficult to interpret without careful consideration of evaluation methodology, since the studies use different metrics on different datasets. This lack of a consistent evaluation framework across the field makes it impossible to determine whether Xuan et al. represent a genuine breakthrough or simply a more favorable experimental setup. The finding from Jeong et al. that reassignment chains are a significant problem also deserves more attention: most studies in this era treated assignment as a one-shot prediction problem, but in practice, incorrect assignments incur real coordination costs. Future work should adopt standardized benchmarks, explicitly model reassignment cost as part of the evaluation, and consider developer availability and workload, factors that none of these studies incorporated.

Theme: Automatically Detecting Duplicate Issues

Observations

- Zou et al. (2016), the landmark paper from the classical ML era on duplicate detection, achieved an F1 score of 83% by combining multiple factors including textual similarity, execution traces, and traditional ML classifiers, rather than relying on text alone.
- The inclusion of execution traces as a feature, meaning runtime information capturing the call stack or program state at the time of the bug, allowed Zou et al. to identify duplicates that were worded very differently but triggered by the same underlying code path.
- Zou et al. also conducted a user study in which developers rated the tool's duplicate suggestions as helpful, providing a measure of practical utility beyond classification metrics alone.

Critique/Implications for Future Research

While Zou et al.'s multi-factor approach represents a meaningful methodological advance, the reliance on execution traces as a feature creates a significant practical limitation: execution traces are only available when users submit crash reports or when automated testing infrastructure captures runtime data. The majority of issues filed in open-source projects arrive as plain natural language descriptions from end users who have no access to execution traces, meaning this approach is not generalizable to the most common issue submission scenario. Additionally, a single study with a user study component, however encouraging, is insufficient to establish the broader applicability of this technique. Future research should investigate how much of the performance advantage from execution traces can be recovered through purely textual or code-structural signals, and should test duplicate detection systems across a wider variety of project types, including those where runtime data is not available.